Lab Assignment - 4

Sequence Alignment

BT3040 - Bioinformatics

Even Semester 2026

Course Instructor: Professor Michael Gromiha

K.Varunkumar | BE23B016

Department of Biotechnology

Indian Institute of Technology, Madras

Question 1

i) The sequences of the β-chain of haemoglobin from both species were obtained from UniProtKB (Entry ID - **P68871** (human), Entry ID - **P02112** (chicken)).

Given below is the code used to generate the box plot for a general sequence alignment.

```
# Question 1
def box_plot(seq1,seq2):

  box_mat = []

  for n2 in seq2:
    temp = ()
    for n1 in seq1:
      if n1 == n2:
        temp = temp + (1,)
      else:
        temp = temp + (0,)

    box_mat.append(temp)
    del temp

  cmap = ListedColormap(["black", "yellow"])

  plt.figure(figsize=(8,6))
  plt.imshow(box_mat, cmap=cmap, origin="lower", interpolation="nearest")

  plt.xlabel("Human -globin")
  plt.ylabel("Chicken -globin")
  plt.title("Dot Plot: Human vs Chicken Hemoglobin  Chain")

  legend_elements = [
    Patch(facecolor='yellow', edgecolor='yellow', label='Match'),
    Patch(facecolor='black', edgecolor='black', label='Mismatch')]

  plt.legend(handles=legend_elements,
             loc='upper left',
             bbox_to_anchor=(1.02, 1),
             borderaxespad=0)

  plt.tight_layout()
  plt.show()

H_HBB="MVHLTPEEKSAVTALWGKVNVDEVGGEALGRLLVVYPWTQRFFESFGDLSTPDAVMGNPKVKAHGKKVLGAFSD
GLAHLDNLKGTFATLSELHCDKLHVDPENFRLLGNVLVCVLAHHFGKEFTPPVQAAYQKVVAGVANALAHKYH"

C_HBB="MVHWTAEEKQLITGLWGKVNVAECGAEALARLLIVYPWTQRFFASFGNLSSPTAILGNPMVRAHGKKVLTSFGD
AVKNLDNIKNTFSQLSELHCDKLHVDPENFRLLGDILIIVLAAHFSKDFTPECQAAWQKLVRVVAHALARKYH"

box_plot(H_HBB, C_HBB)
```

The output generated is given below

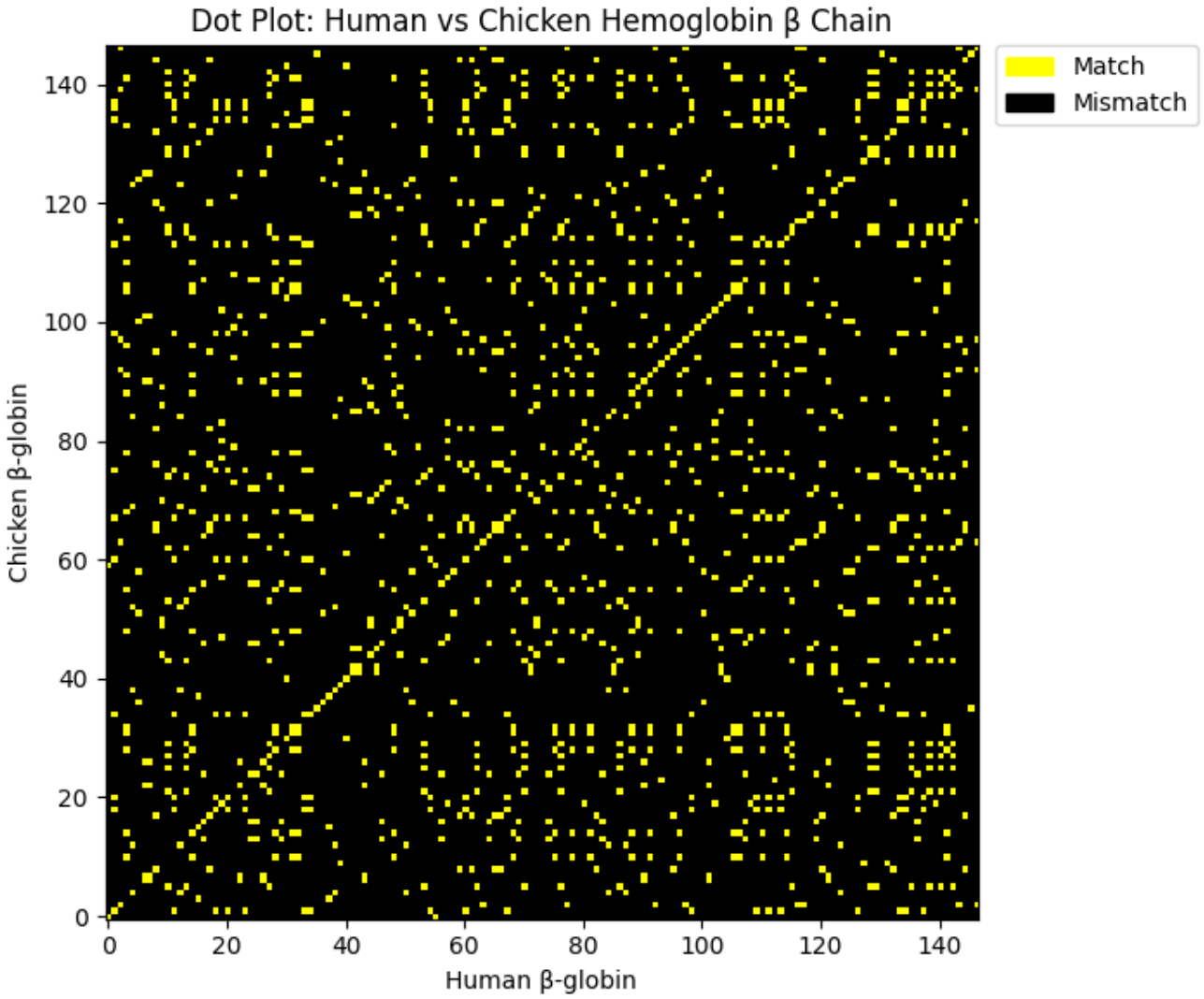

Figure 1.1: Dot-plot (alignment of human and chicken hemoglobin β-chain)

```
def aligned_segments(seq1, seq2):
    alignments = []
    start = None

    for n in range(min(len(seq1),len(seq2))):
        if seq1[n] == seq2[n]:
            if start is None:
                start = n
        else:
            if start is not None:
                alignments.append((start + 1, n))
                start = None
    if start is not None:
        alignments.append((start + 1, min(len(seq1), len(seq2))))

    pprint.pprint(alignments)

aligned_segments(H_HBB, C_HBB)
```

The following regions were found to be the same in both the sequences:

Residues 1-3	Residue 52	Residues 89-108
Residue 5	Residue 54	Residue 111
Residues 7-9	Residues 57-59	Residues 114-116
Residue 13	Residue 61	Residue 118-119
Residues 15-21	Residues 63-69	Residue 121
Residue 23	Residues 72-72	Residues 123-125
Residue 25	Residue 74-74	Residues 128-130
Residues 27-29	Residues 79-81	Residues 132-133
Residues 31-33	Residues 83-83	Residue 135
Residues 35-43	Residue 85-86	Residues 138-139
Residues 45-47	Residue 85-86	Residues 141-143
Residues 49-50	Residue 85-86	Residues 145-147

---

ii) The manual dot-plot of the first 20 residues of both the proteins was made using Excel, and compared with the program as shown below:

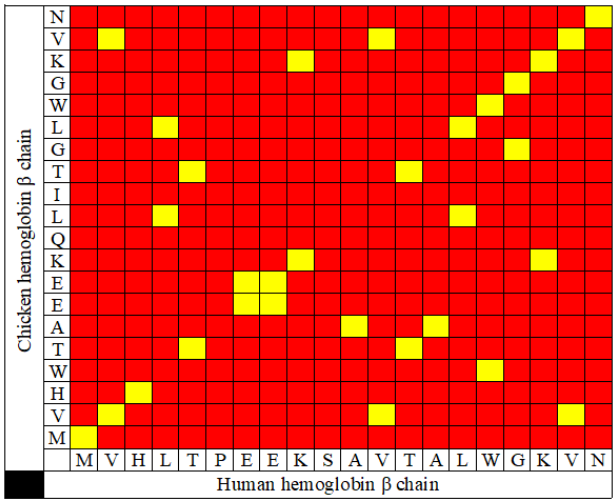

Figure 1.2.1: Manual dot-plot using Excel

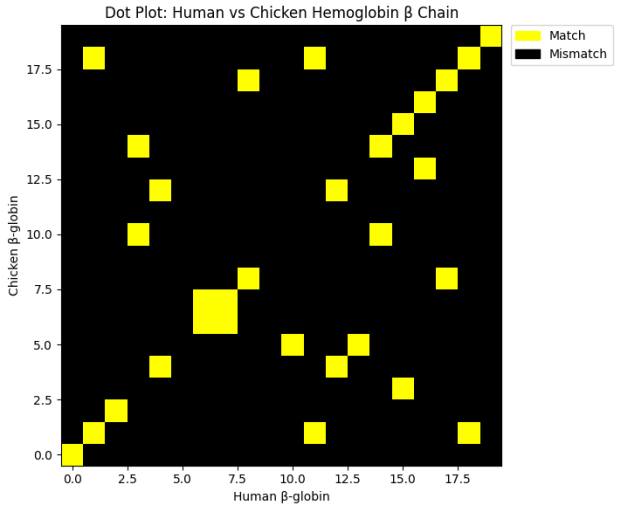

Figure 1.2.2: Validation using the program

Question 2

```
# Question 2
seq1, seq2 = "AATCTATA", "AAG--ATA"

scores = {
    "match": 1,
    "mismatch": 0,
    "op": -2,
    "gp": -1}
```

```
score, n, Org = 0, 0, False

for n in range(len(seq1)):

  if seq1[n] == "-" or seq2[n] == "-":

    if not Org:
      score += scores["op"]
      Org = True

    score += scores["gp"]

  elif seq1[n] == seq2[n]:
    Org = False
    score += scores["match"]

  else:
    Org = False
    score += scores["mismatch"]

print("Score =",score)
```

The obtained output of the program is given below:

```
Score = 1
```

Question 3

The manual verification of the score calculation is given below:

Sequence 1	A	A	T	C	T	A	T	A
Sequence 2	A	A	G	-	-	A	T	A
Match / Mismatch & Gap penalties	+1	+1	0	-1	-1	+1	+1	+1
Origination penalty	0	0	0	-2	0	0	0	0
Total Score	5 - 4 = 1							

Figure 3.1: Manual Verification of Question 2 Score

The score of the given alignment is verified to be 1.

Question 4

```
# Question 4
def NW(seq1,seq2):

  scores = {
    "match": 2,
    "mismatch": -1,
    "gap": -2}

  n,m = len(seq1),len(seq2)
  mat = np.zeros((n+1, m+1))

  for row in range(n+1):
    mat[row][0] = row*scores["gap"]

  for col in range(m+1):
    mat[0][col] = col*scores["gap"]

  for row in range(1,n+1):
    for col in range(1,m+1):
      if seq1[row-1] == seq2[col-1]:
        mat[row][col] = max(mat[row-1][col]+scores["gap"], mat[row][col-1]
        +scores["gap"], mat[row-1][col-1]+scores["match"])
      else:
        mat[row][col] = max(mat[row-1][col]+scores["gap"], mat[row][col-1]
        +scores["gap"], mat[row-1][col-1]+scores["mismatch"])

  brow,bcol = n,m
  matched_1 = ""
  matched_2 = ""

  while brow > 0 and bcol > 0:

    if mat[brow-1][bcol] + scores["gap"] == mat[brow][bcol]:
      matched_1 += seq1[brow-1]
      matched_2 += "_"
      brow -= 1

    elif mat[brow][bcol-1] + scores["gap"] == mat[brow][bcol]:
      matched_1 += "_"
      matched_2 += seq2[bcol-1]
      bcol -= 1

    else:
      matched_1 += seq1[brow-1]
      matched_2 += seq2[bcol-1]
      brow -= 1
      bcol -= 1

  matched_1 = matched_1[::-1]
  matched_2 = matched_2[::-1]
```

```
  if brow!=0:
    matched_1 = seq1[:brow] + matched_1
    matched_2 = "_"*brow + matched_2
  elif bcol!=0:
    matched_1 = "_"*bcol + matched_1
    matched_2 = seq2[:bcol] + matched_2
  print(mat)
  print("Score =", mat[n][m])
  print(matched_1)
  print(matched_2)

s1,s2 = "ACAGTCGAACG", "ACCGTCCG"
NW(s1,s2)
```

The obtained output of the program is given below:

```
[[  0.  -2.  -4.  -6.  -8. -10. -12. -14. -16.]
 [ -2.   2.   0.  -2.  -4.  -6.  -8. -10. -12.]
 [ -4.   0.   4.   2.   0.  -2.  -4.  -6.  -8.]
 [ -6.  -2.   2.   3.   1.  -1.  -3.  -5.  -7.]
 [ -8.  -4.   0.   1.   5.   3.   1.  -1.  -3.]
 [-10.  -6.  -2.  -1.   3.   7.   5.   3.   1.]
 [-12.  -8.  -4.   0.   1.   5.   9.   7.   5.]
 [-14. -10.  -6.  -2.   2.   3.   7.   8.   9.]
 [-16. -12.  -8.  -4.   0.   1.   5.   6.   7.]
 [-18. -14. -10.  -6.  -2.  -1.   3.   4.   5.]
 [-20. -16. -12.  -8.  -4.  -3.   1.   5.   3.]
 [-22. -18. -14. -10.  -6.  -5.  -1.   3.   7.]]
Score = 7.0
ACAGTCGAACG
ACCGTC___CG
```

Question 5

	-	A	C	C	G	T	C	C	G
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2	0	-2	-4	-6	-8	-10	-12
C	-4	0	4	2	0	-2	-4	-6	-8
A	-6	-2	2	3	1	-1	-3	-5	-7
G	-8	-4	0	1	5	3	1	-1	-3
T	-10	-6	-2	-1	3	7	5	3	1
C	-12	-8	-4	0	1	5	9	7	5
G	-14	-10	-6	-2	2	3	7	8	9
A	-16	-12	-8	-4	0	1	5	6	7
A	-18	-14	-10	-6	-2	-1	3	4	5
C	-20	-16	-12	-8	-4	-3	1	5	3
G	-22	-18	-14	-10	-6	-5	-1	3	7

Figure 5.1: Constructing the Table

	-	A	C	C	G	T	C	C	G
-	0	-2	-4	-6	-8	-10	-12	-14	-16
A	-2	2	0	-2	-4	-6	-8	-10	-12
C	-4	0	4	2	0	-2	-4	-6	-8
A	-6	-2	2	3	1	-1	-3	-5	-7
G	-8	-4	0	1	5	3	1	-1	-3
T	-10	-6	-2	-1	3	7	5	3	1
C	-12	-8	-4	0	1	5	9	7	5
G	-14	-10	-6	-2	2	3	7	8	9
A	-16	-12	-8	-4	0	1	5	6	7
A	-18	-14	-10	-6	-2	-1	3	4	5
C	-20	-16	-12	-8	-4	-3	1	5	3
G	-22	-18	-14	-10	-6	-5	-1	3	7

Figure 5.2: Backtracking

Question 6

```
# Question 6
def SW(seq1,seq2):

  scores = {
    "match": 2,
    "mismatch": -1,
    "gap": -2}


  n = len(seq1)
  m = len(seq2)

  mat = np.zeros((n+1, m+1))
  max_loc, greatest = (0,0),0

  for row in range(n+1):
    mat[row][0] = 0

  for col in range(m+1):
    mat[0][col] = 0

  for row in range(1,n+1):
    for col in range(1,m+1):
      if seq1[row-1] == seq2[col-1]:
        mat[row][col] = max(mat[row-1][col]+scores["gap"], mat[row][col-1]
        +scores["gap"], mat[row-1][col-1]+scores["match"],0)
      else:
        mat[row][col] = max(mat[row-1][col]+scores["gap"], mat[row][col-1]
        +scores["gap"], mat[row-1][col-1]+scores["mismatch"],0)

      if mat[row][col] >= greatest:
        greatest = mat[row][col]
        max_loc  =(row,col)

  brow,bcol = max_loc
  matched_1 = ""
  matched_2 = ""

  while mat[brow][bcol] > 0:

    if mat[brow-1][bcol] + scores["gap"] == mat[brow][bcol]:
      matched_1 += seq1[brow-1]
      matched_2 += "_"
      brow -= 1

    elif mat[brow][bcol-1] + scores["gap"] == mat[brow][bcol]:
      matched_1 += "_"
      matched_2 += seq2[bcol-1]
      bcol -= 1
```

```
    else:
      matched_1 += seq1[brow-1]
      matched_2 += seq2[bcol-1]
      brow -= 1
      bcol -= 1


  matched_1 = matched_1[::-1]
  matched_2 = matched_2[::-1]


  print(mat)
  print("Score =", greatest)
  print(matched_1)
  print(matched_2)

seq1, seq2 = "ACGTATCGCGTATA", "GATGCGTATCG"
SW(seq1,seq2)
```

The obtained output of the program is given below:

```
[[ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  2.  0.  0.  0.  0.  0.  2.  0.  0.  0.]
 [ 0.  0.  0.  1.  0.  2.  0.  0.  0.  1.  2.  0.]
 [ 0.  2.  0.  0.  3.  1.  4.  2.  0.  0.  0.  4.]
 [ 0.  0.  1.  2.  1.  2.  2.  6.  4.  2.  0.  2.]
 [ 0.  0.  2.  0.  1.  0.  1.  4.  8.  6.  4.  2.]
 [ 0.  0.  0.  4.  2.  0.  0.  3.  6. 10.  8.  6.]
 [ 0.  0.  0.  2.  3.  4.  2.  1.  4.  8. 12. 10.]
 [ 0.  2.  0.  0.  4.  2.  6.  4.  2.  6. 10. 14.]
 [ 0.  0.  1.  0.  2.  6.  4.  5.  3.  4.  8. 12.]
 [ 0.  2.  0.  0.  2.  4.  8.  6.  4.  2.  6. 10.]
 [ 0.  0.  1.  2.  0.  2.  6. 10.  8.  6.  4.  8.]
 [ 0.  0.  2.  0.  1.  0.  4.  8. 12. 10.  8.  6.]
 [ 0.  0.  0.  4.  2.  0.  2.  6. 10. 14. 12. 10.]
 [ 0.  0.  2.  2.  3.  1.  0.  4.  8. 12. 13. 11.]]
Score = 14.0
ATCGCGTAT
AT_GCGTAT
```

Link to the google drive containing the python code and Excel sheets used in this assignment: `https://drive.google.com/drive/u/0/folders/169BqRA5MQfmesZVib19pNbBXWxz_tOf5`

Annotations have been omitted from the submitted PDF for the sake of brevity. The fully annotated version of the code is available in the code folder within the attached Google Drive.